Import Data using Transform Maps (Spreadsheet)

Project Documentation Format

Date:	30 September 202
Team ID:	6ab4d1f60fc666a751b533bb
Project Name:	Import Data using Transform Maps using Spreadsheet.”
Maximum Member:	5
Team Leader:	muthu lakshmi.N
Member:	abi.J ,alagu nachiyar.S,Jesu snowrina.A,Muthu meenakshi.m

1. Introduction

- **Project Title:** Import Data using Transform Maps (Spreadsheet)
- **Team Members:** List team members and their specific development roles.

2. Project Overview

- **Purpose:** Briefly describe the business goals, challenges solved, and data objectives when using a spreadsheet to stage and transform records safely into Target Tables.
- **Features:** Highlight critical capabilities including custom mapping schemes, automated data coercion, field validations, status checks, error handlers, and successful bulk records creation logs.

3. Architecture

- **Frontend / Staging Setup:** Describe the UI layout where spreadsheets are loaded, field attributes are previewed, and mapping adjustments are visually structured.
- **Backend / Transformation Logic:** Outline the processing pipeline, execution engines, or script maps responsible for pulling staging rows and converting values.
- **Database / Target Entities:** Detail the structural target schemas, relational keys, required lookups, database indexes, and structural fields modified by incoming records.

4. Setup Instructions

- **Prerequisites:** List software requirements, API profiles, platform access controls, staging plugins, or execution modules necessary to run the engine.

- **Installation:** Provide a clear step-by-step roadmap to install integration tools, configure connection endpoints, verify field lengths, and provision system environment flags.

5. Folder Structure

- **Client / Workspace Layout:** Explain the UI hierarchy, import files view components, metadata templates, and tracking tables layout.
- **Server / Integration Engine:** Outline the backend organization handling raw parser endpoints, field mapping handlers, exception frameworks, and transformation matrices.

6. Running the Application

- **Staging Activation:** Provide actionable commands or execution parameters to start the input processor engine or launch your staging web panel locally.
- **Transform Execution:** Detail explicit triggers, script commands, or process buttons needed to activate live bulk mapping runs across the system backend.

7. API Documentation

- **Data Endpoints:** Exhaustively document all backend API services managing incoming record packets and staging pools.
- **Payload Schema:** Provide exact HTTP request verbs (e.g., POST), parameter filters, source data headers, and full status code JSON response structures.

8. Authentication

- **Access Protocols:** Explain how connection safety is secured, authorized identities are processed, and restricted file fields are isolated via encryption layers.
- **Tokens & Context:** Include functional token frameworks, operational cookies, credentials storage rules, or system keys leveraged during transaction processing.

9. User Interface

- **Interface Walkthrough:** Provide crisp screenshots, structural layout flowcharts, or step GIFs tracking source column mappings, staging views, and execution panels.

10. Testing

- **Testing Strategy:** Describe verification tools, boundary case setups, dry-run routines, mock spreadsheet parsing checks, and accuracy benchmarks.
- **Performance Thresholds:** Detail model validation limits, strict data constraints, matching field verifications, and stress bounds for large-volume batch arrays.

11. Screenshots or Demo

- **Visual Evidence:** Provide high-fidelity screenshots showcasing mapped data points, successful bulk job processing dashboards, or staging error modals.

- **Deployment Reference:** Supply safe links to functional staging environments, product sandbox builds, or interactive configuration mockups.

12. Known Issues

- **Identified Limitations:** Document active boundary limitations, file system locking bugs, unmapped field omissions, or specific Excel attribute translation traps.
- **Handling Anomalies:** Provide immediate operational overrides when encounter error signals like 'Error Extracting Field Attributes' during execution loops.

13. Future Enhancements

- **Automated Layout Mappings:** Outline dynamic structural updates using intelligent fuzzy logic models to match disparate column headers seamlessly without manual steps.
- **Real-Time Exception Resolvers:** Propose next-generation inline interactive correction prompts allowing users to map standalone anomalies directly mid-execution.